String Searching Algorithms

- Spell Checkers
- Spam Filters
- Intrusion Detection System
- Search Engines Plagiarism Detection Bioinformatics,
- Digital Forensics and Information Retrieval Systems and etc.

Spell Checkers

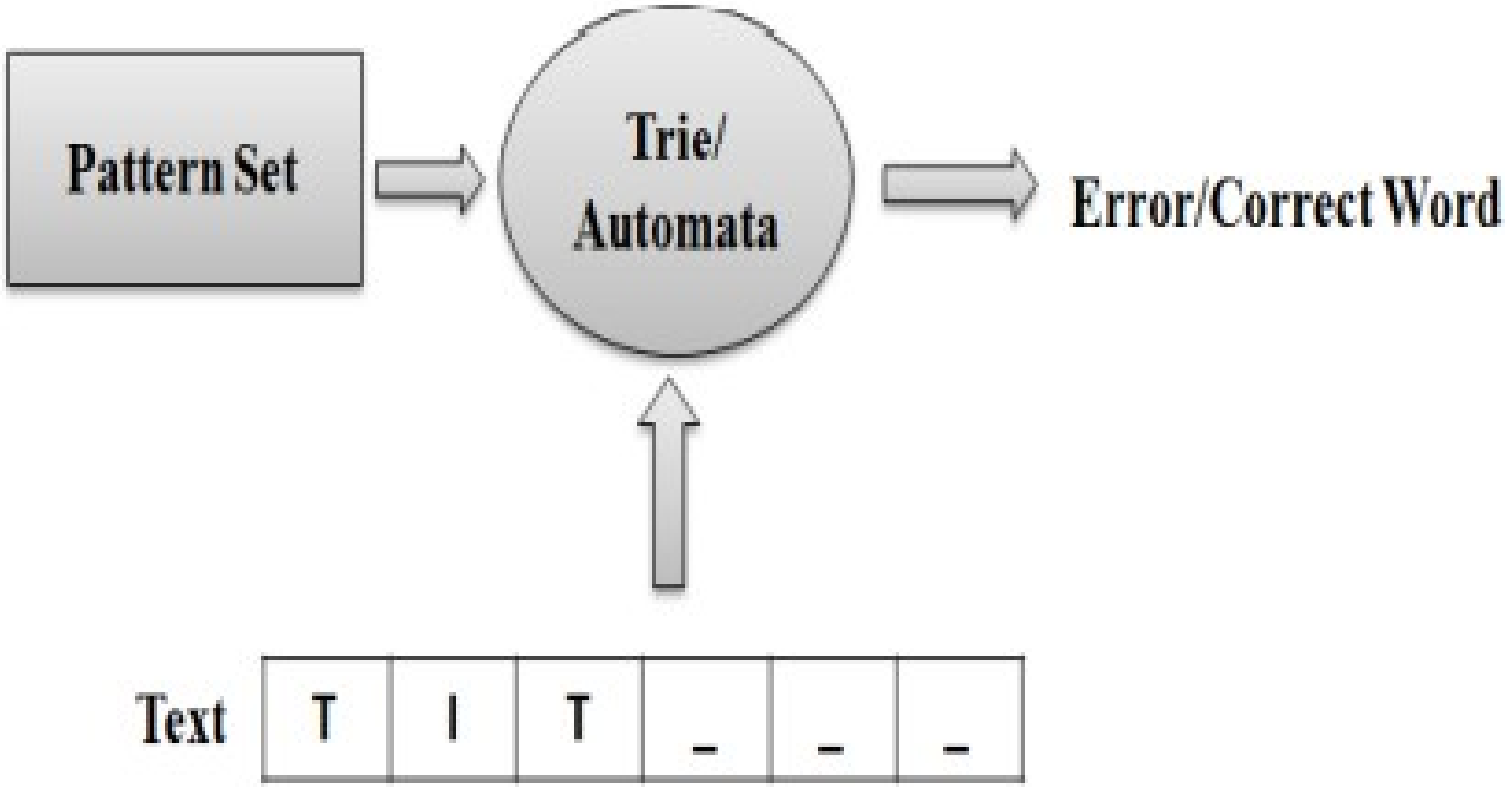


Figure Spell Checker

Spam Filters / Spam Detection Systems

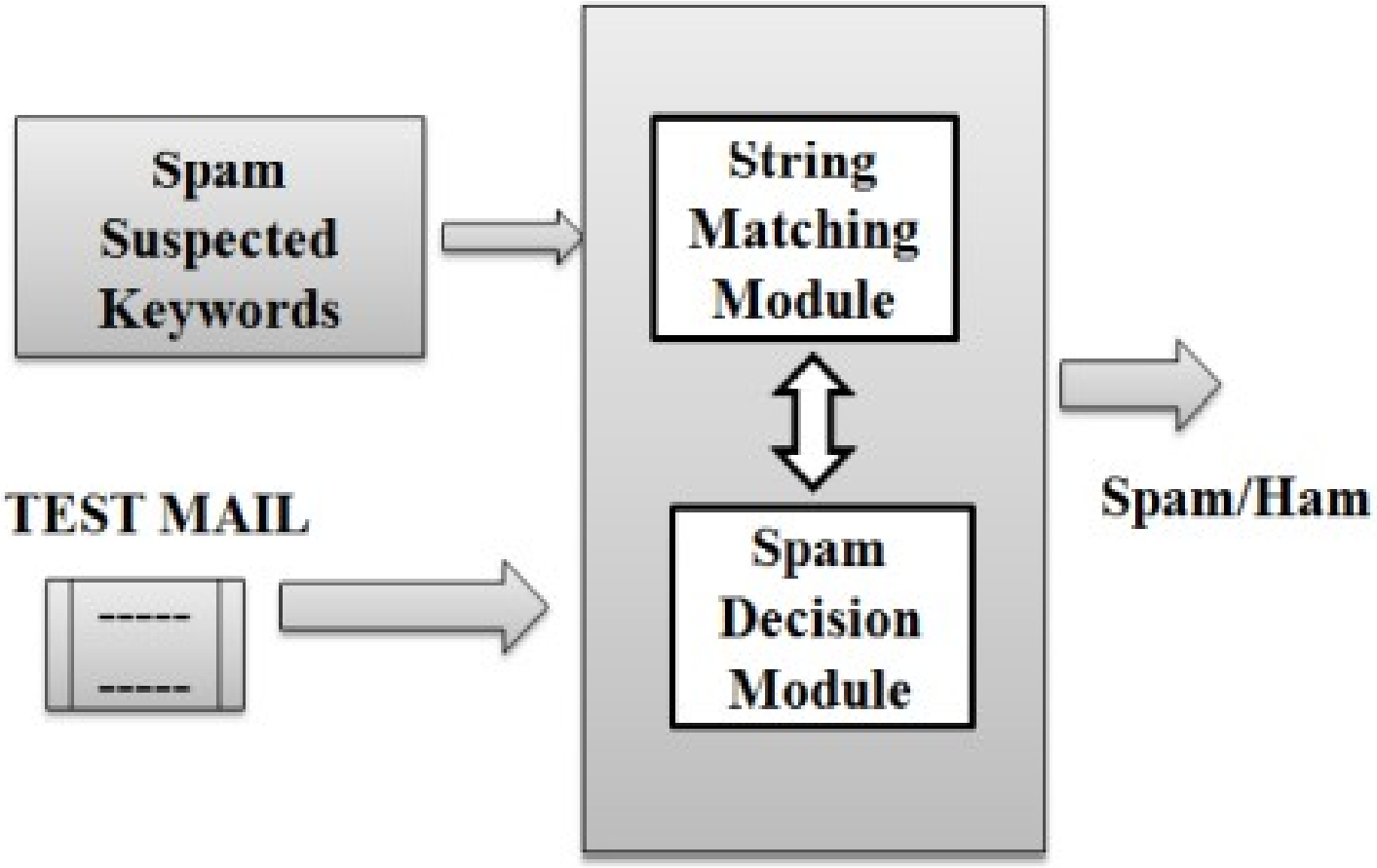


Figure : Spam Filter

Intrusion Detection System

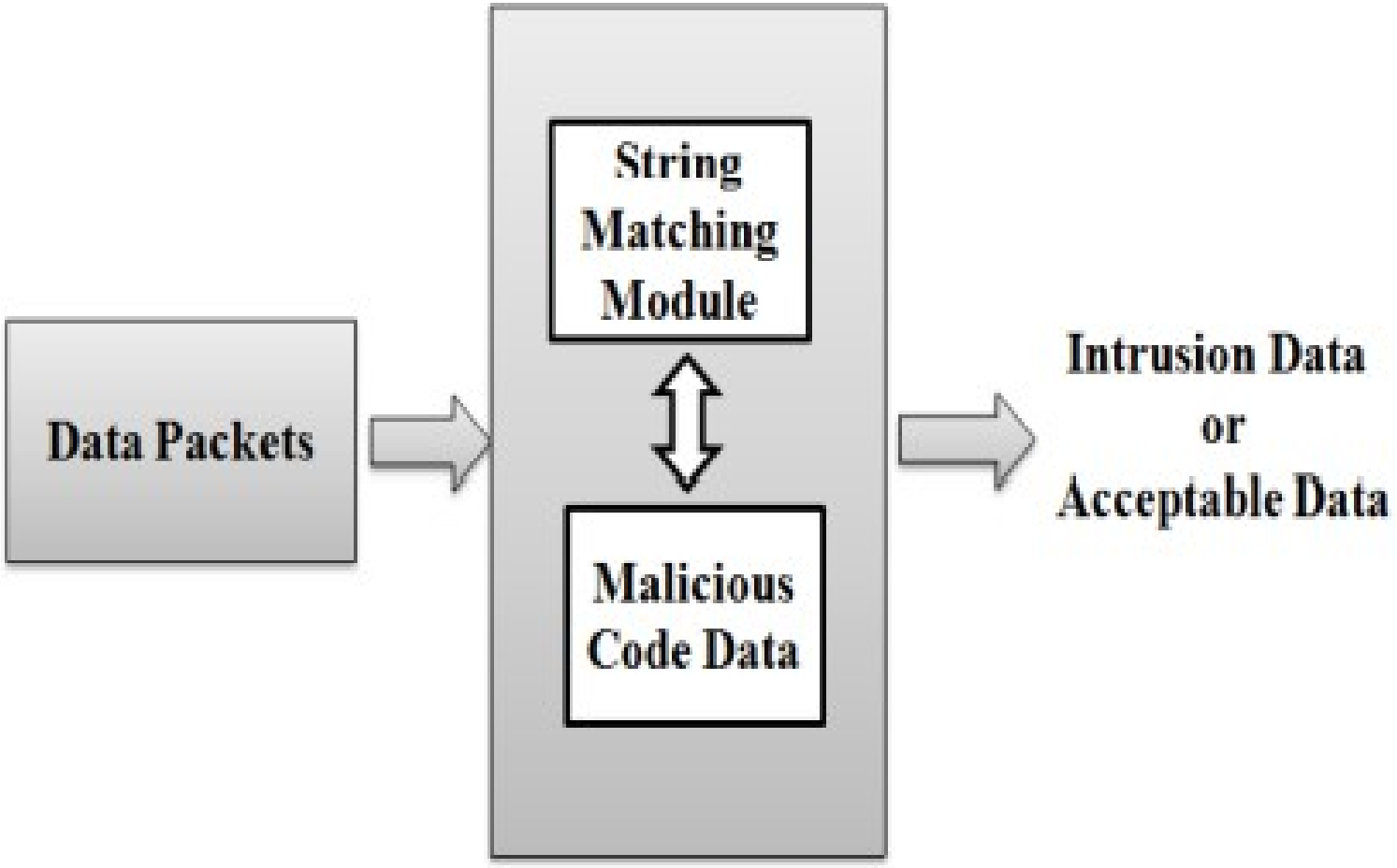


Figure : Intrusion Detection System Model

Plagiarism Detection

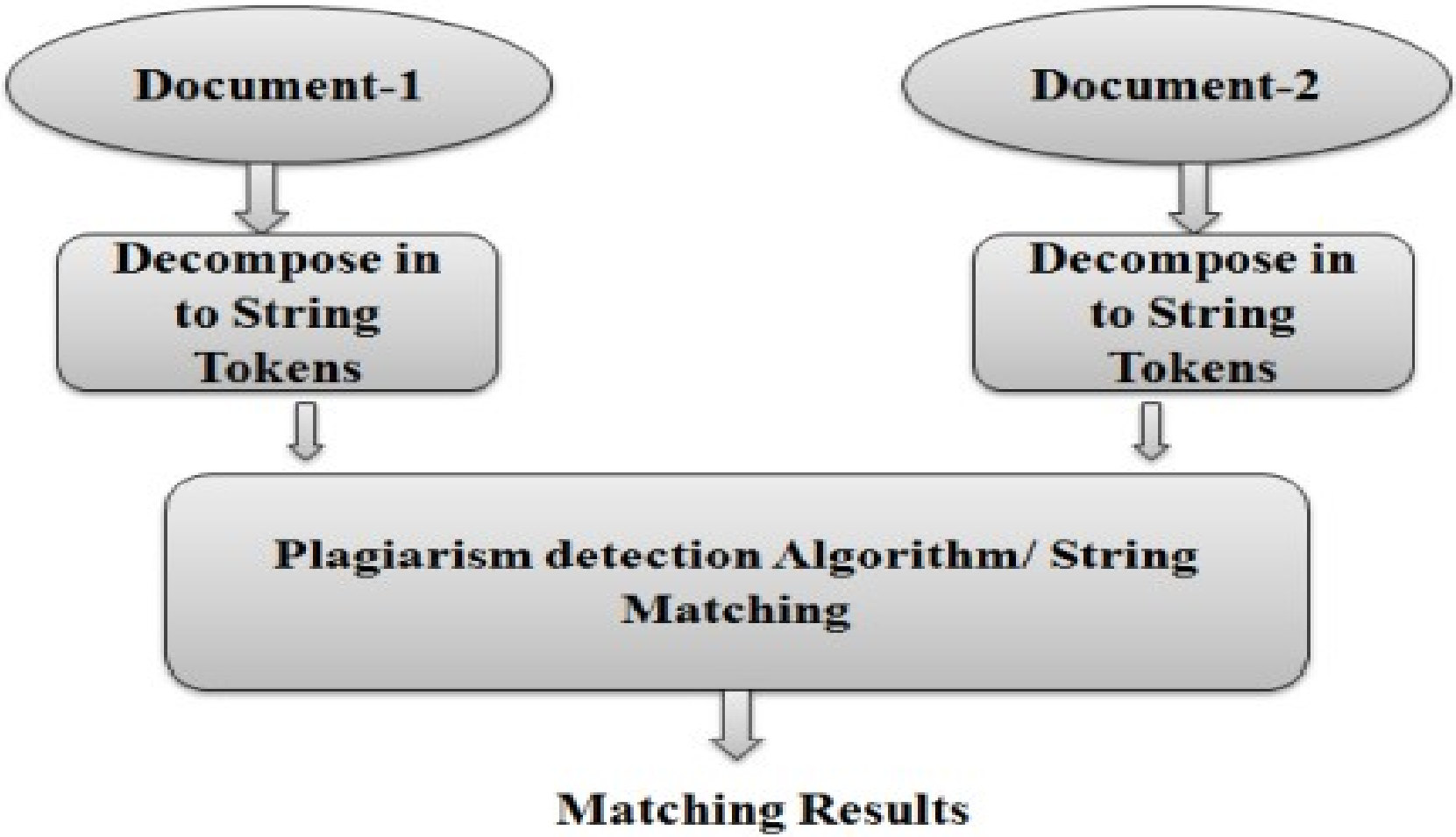


Figure : Plagiarism Detection System

Search Engines / Content Searching in Large Databases

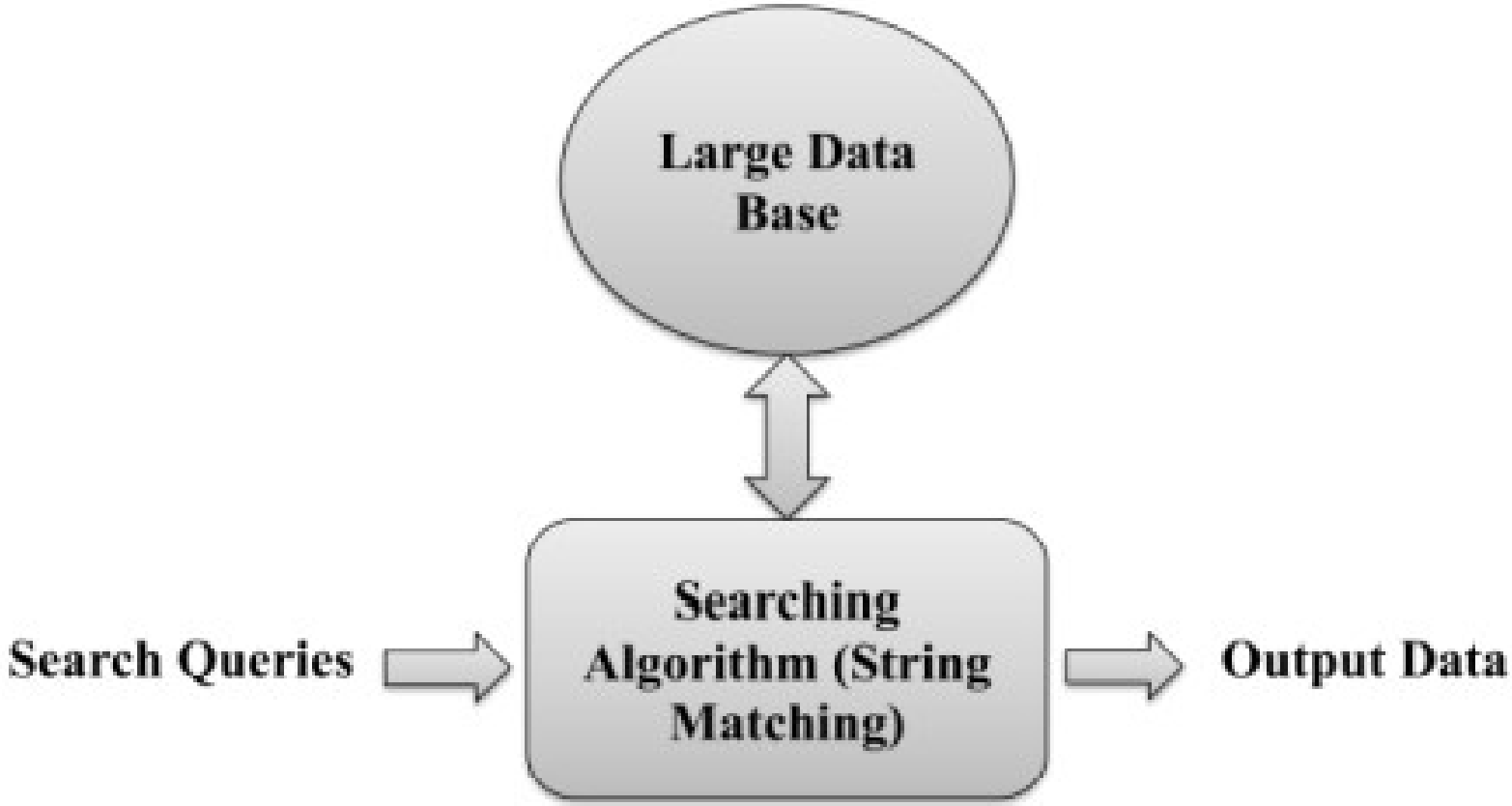


Figure : Search Engine Module

Bioinformatics / DNA Sequencing

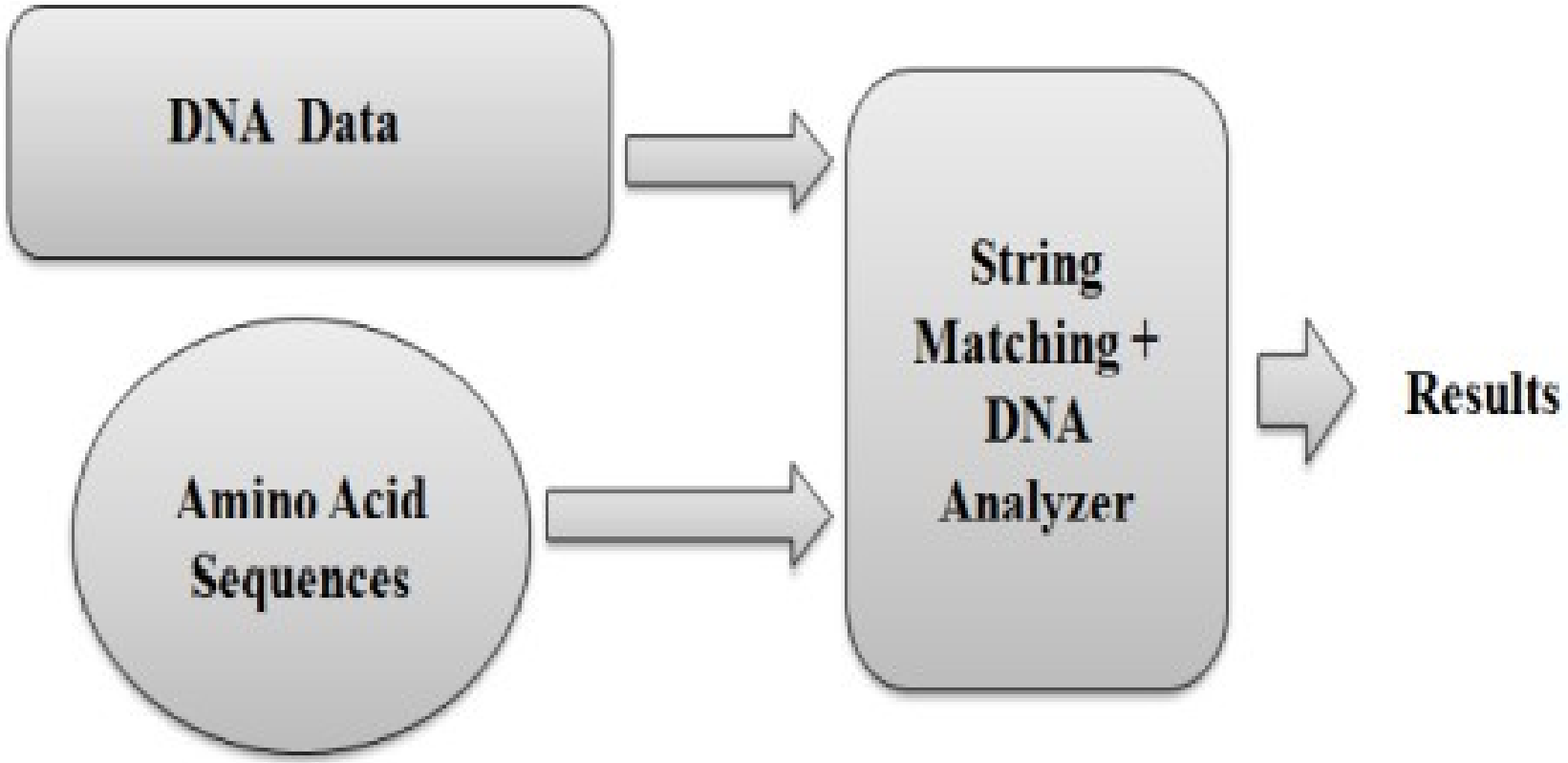


Figure : DNA Sequencing Module

Digital Forensics

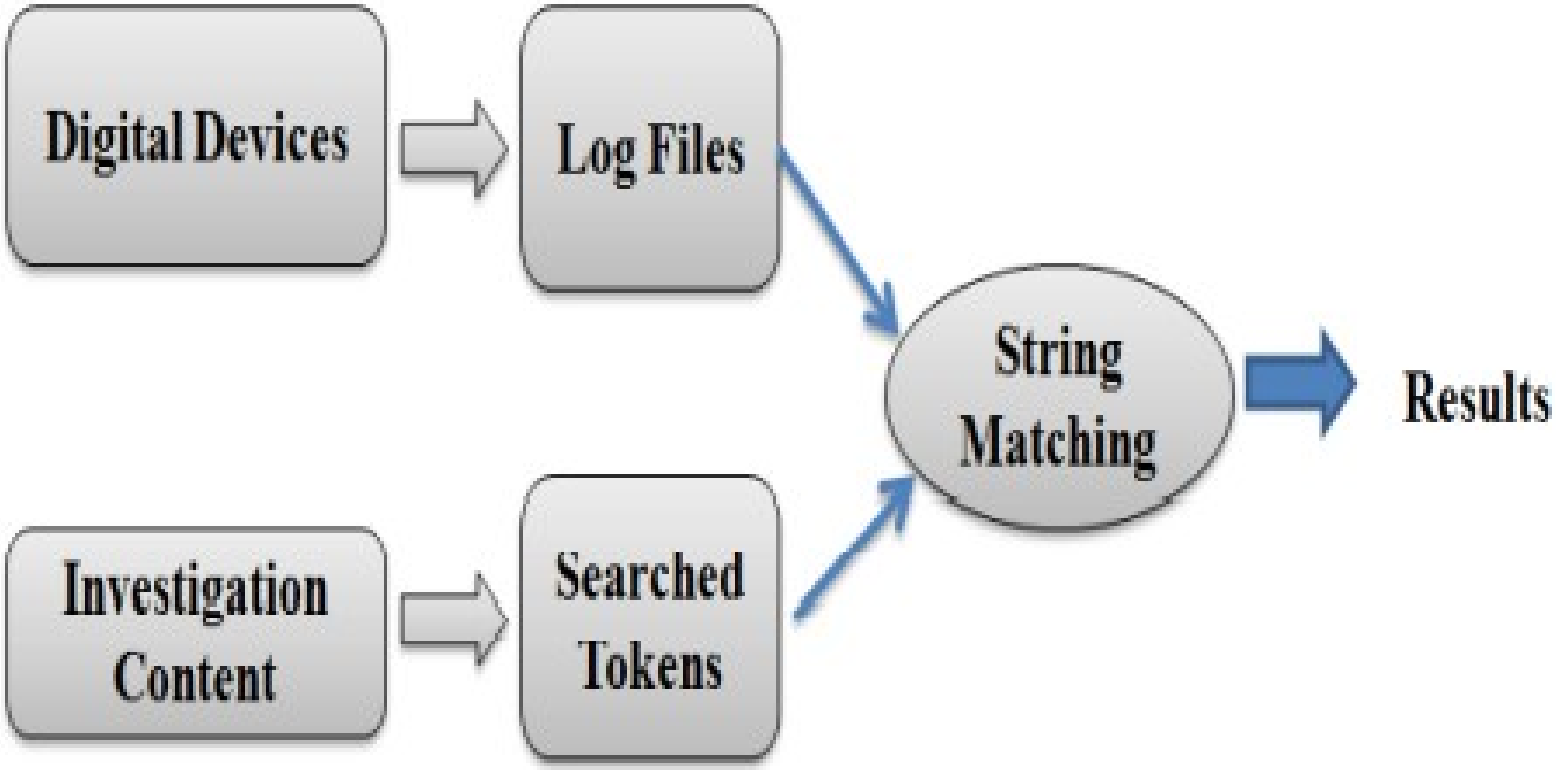


Figure : Digital Forensic Results

Information Retrieval

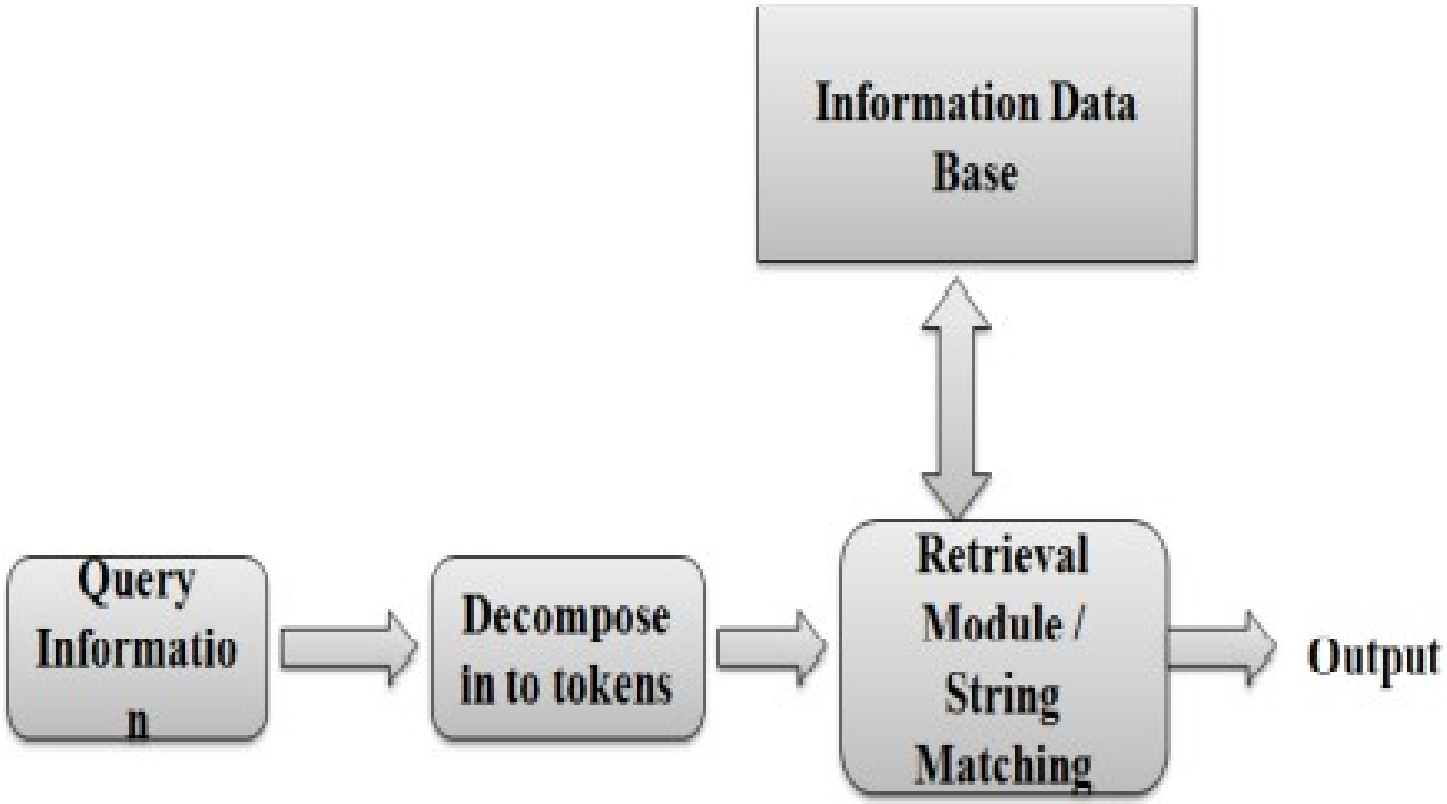
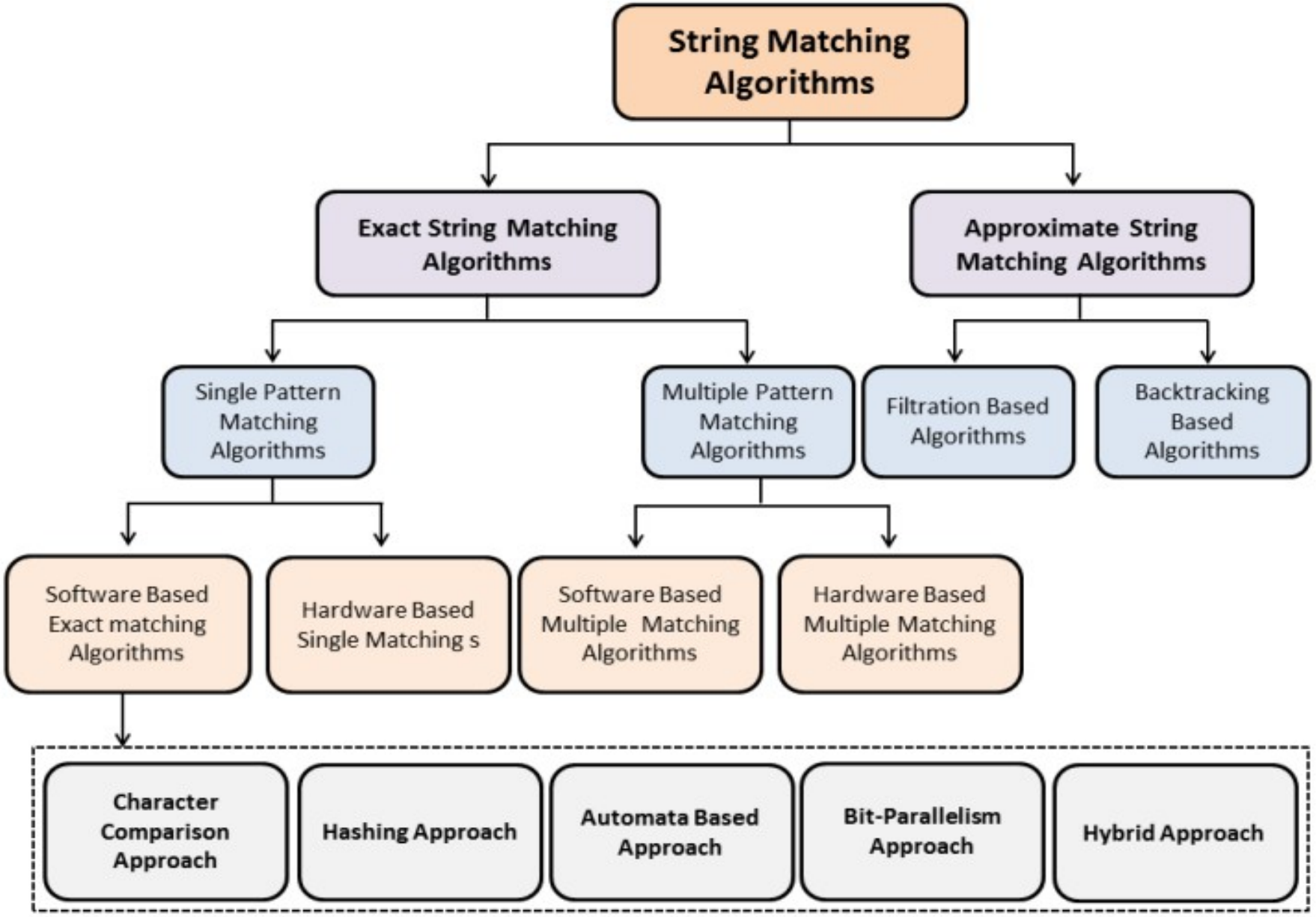


Figure : Information Retrieval Modal

Classification of String Matching Algorithms



String matching: Find one, or more generally, all the occurrences of a pattern $x = [x_0 x_1 .. x_{m-1}]$; $x_i \in \Sigma$; $i = 0, \ldots, m-1$, in a text (string) $y = [y_0 y_1 .. y_{n-1}]$; $y_j \in \Sigma$; $j = 0, \ldots, n-1$

- Two basic variants:
  1. Given a pattern, find its occurrences in any initially unknown text
     - Solutions by preprocessing the pattern using finite automata models or combinatorial properties of strings
  2. Given a text, find occurrences of any initially unknown pattern
     - Solutions by indexing the text with the help of trees or finite automata

- Sliding window: Scan the text by a window of size, which is generally equal to m
- An attempt: Align the left end of the window with the text and compare the characters in the window with those of the pattern
  - Each attempt (step) is associated with position j in the text when the window is positioned on $y_j..y_{j+m-1}$
- Shift the window to the right after the whole match of the pattern or after a mismatch

Effectiveness of the search depends on the order of comparisons:

1. The order is not relevant (e.g. naïve, or brute-force algorithm)
2. The natural left-to-right order (the reading direction)
3. The right-to-left order (the best algorithms in practice)
4. A specific order (the best theoretical bounds)

Single – Pattern Matching

Notation:

m – the length (size) of the pattern; n – the length of the searched text

String search algorithm	Time complexity for	
	preprocessing	matching
Naïve	0 (none)	$\Theta(n \cdot m)$
Rabin-Karp	$\Theta(m)$	avg $\Theta(n+m)$ worst $\Theta(n \cdot m)$
Finite state automaton	$\Theta(m\|\Sigma\|)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$
Boyer-Moore	$\Theta(m+\|\Sigma\|)$	$\Omega(n/m)$, $O(n)$
Bit based (approximate)	$\Theta(m+\|\Sigma\|)$	$\Theta(n)$

String Searching

- The previous slide is not a great example of what is meant by "String Searching." Nor is it meant to ridicule people without eyes....
- The object of string searching is to find the location of a specific text pattern within a larger body of text (e.g., a sentence, a paragraph, a book, etc.).
- As with most algorithms, the main considerations for string searching are speed and efficiency.
- There are a number of string searching algorithms in existence today, but the two we shall review are Brute Force and Rabin-Karp.

Few variables

- n: the length of the text
- m: the length of the pattern(string)
- C_n: the expected number of comparisons performed by an algorithm while searching the pattern in a text of length n

Naive Method – Brute - Force

```
for ( j = 0; j <= n - m; j++ ) {
    for ( i = 0; i < m && x[i] == y[i + j]; i++ );
    if ( i >= m ) return j;
}
```

Main features of this easy (but slow) $O(nm)$ algorithm:

- No preprocessing phase
- Only constant extra space needed
- Always shifts the window by exactly 1 position to the right
- Comparisons can be done in any order
- mn expected text characters comparisons

Pattern: abaa; *searched string*: ababbaabaaab

```
ababbaabaaab                ............

abaa________  step 1        ABA#         mismatch: 4th letter
_abaa_______  step 2        _#...        mismatch: 1st letter
__abaa______  step 3        __AB#.       mismatch: 3rd letter
___abaa_____  step 4        ___#...      mismatch: 1st letter
____abaa____  step 5        ____#...     mismatch: 1st letter
_____abaa___  step 6        _____A#..    mismatch: 2nd letter
______abaa__  step 7        ______ABAA   success
_______abaa_  step 8        _______#...  mismatch: 1st letter
________abaa  step 9        ________.#..mismatch: 2nd letter
```

Runs with 9 window steps and 18 character comparisons

Brute Force

- The Brute Force algorithm compares the pattern to the text, one character at a time, until unmatching characters are found:

```
TWO ROADS DIVERGED IN A YELLOW WOOD
ROADS
```

```
TWO ROADS DIVERGED IN A YELLOW WOOD
 ROADS
```

```
TWO ROADS DIVERGED IN A YELLOW WOOD
  ROADS
```

```
TWO ROADS DIVERGED IN A YELLOW WOOD
   ROADS
```

```
TWO ROADS DIVERGED IN A YELLOW WOOD
    ROADS
```

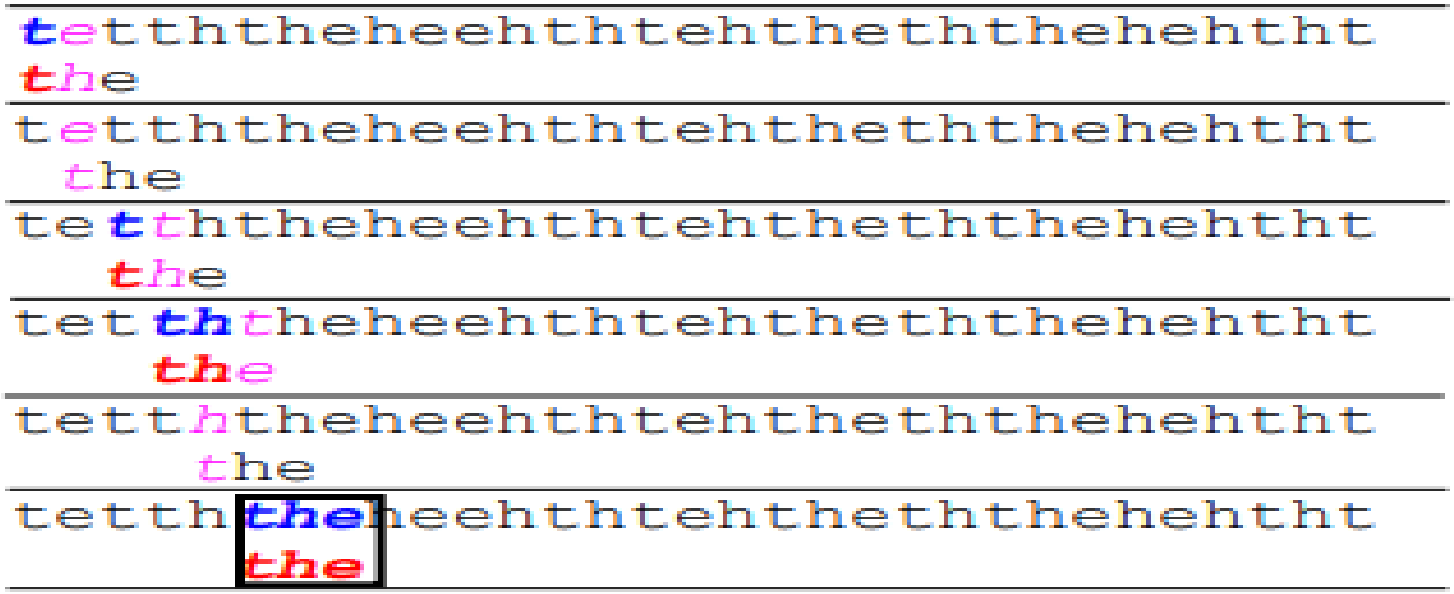
  - Compared characters are italicized.
  - Correct matches are in boldface type.

- The algorithm can be designed to stop on either the first occurrence of the pattern, or upon reaching the end of the text.

Brute Force Pseudo-Code

- Here's the pseudo-code

```
do
  if (text letter == pattern letter)
    compare next letter of pattern to next
    letter of text
  else
    move pattern down text by one letter
while (entire pattern found or end of text)
```



Brute Force-Complexity

- Given a pattern M characters in length, and a text N characters in length…
- **Worst case:** compares pattern to each substring of text of length M. For example, M=5.

1) *AAAAA*AAAAAAAAAAAAAAAAAAAAAAH
   AAAAH **5 comparisons made**
2) *AAAAA*AAAAAAAAAAAAAAAAAAAAAAH
    AAAAH **5 comparisons made**
3) *AAAAA*AAAAAAAAAAAAAAAAAAAAAAH
      AAAAH **5 comparisons made**
4) *AAAAA*AAAAAAAAAAAAAAAAAAAAAAH
       AAAAH **5 comparisons made**
5) *AAAAA*AAAAAAAAAAAAAAAAAAAAAAH
         AAAAH **5 comparisons made**
   ….
N) AAAAAAAAAAAAAAAAAAAAAAAAAA*AAAAH*
                5 comparisons made *AAAAH*

- Total number of comparisons: M (N-M+1)
- Worst case time complexity: O(MN)

Brute Force-Complexity(cont.)

- Given a pattern M characters in length, and a text N characters in length...
- **Best case if pattern found**: Finds pattern in first M positions of text. For example, M=5.

1) *AAAAA*AAAAAAAAAAAAAAAAAAAAAAAAH
   AAAAA **5 comparisons made**

- Total number of comparisons: M
- Best case time complexity: O(M)

Brute Force-Complexity(cont.)

- Given a pattern M characters in length, and a text N characters in length...
- **Best case if pattern not found:** Always mismatch on first character. For example, M=5.

1) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
   OOOOH **1 comparison made**
2) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
    OOOOH **1 comparison made**
3) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
     OOOOH **1 comparison made**
4) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
      OOOOH **1 comparison made**
5) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
       OOOOH **1 comparison made**

...

N) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
1 comparison made OOOOH

- Total number of comparisons: N
- Best case time complexity: O(N)

Knuth-Morris-Pratt Algorithm

Searches for occurrences of a pattern x within a main text string y by employing the simple observation: *after a mismatch, the word itself allows us to determine where to begin the* **next match** *to bypass re-examination of previously matched characters*

- Preprocessing phase: $O(m)$ space and time complexity
- Searching phase: $O(n+m)$ time complexity (independent from the alphabet size)
- At most $2n-1$ character comparisons during the text scan
- The maximum number of comparisons for a single text character: $\leq \log_\eta m$ where $\eta = \frac{1+\sqrt{5}}{2}$ is the golden ratio

The algorithm was invented in 1977 by Knuth and Pratt and independently by Morris, but the three published it jointly

The Knuth-Morris-Pratt Algorithm

- The Knuth-Morris-Pratt (KMP) string searching algorithm differs from the brute-force algorithm by keeping track of information gained from previous comparisons.
- A failure function (f) is computed that indicates how much of the last comparison can be reused if it fais.
- Specifically, f is defined to be the longest prefix of the pattern P[0,..,j] that is also a suffix of P[1,..,j]
  - Note: **not** a suffix of P[0,..,j]
- Example:
  - value of the KMP failure function:

j	0	1	2	3	4	5
$P[j]$	a	b	a	b	a	c
$f(j)$	0	0	1	2	3	0

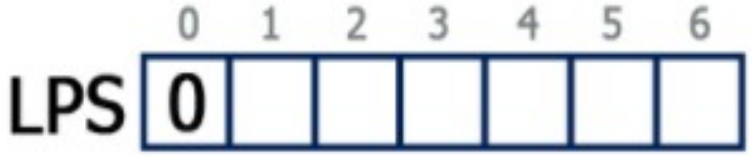
- This shows how much of the beginning of the string matches up to the portion immediately preceding a failed comparison.
  - if the comparison fails at (4), we know the a,b in positions 2,3 is identical to positions 0,1

Example for creating KMP Algorithm's LPS Table (Prefix Table)

Consider the following Pattern

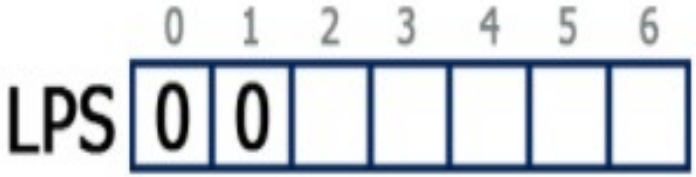
Let us define LPS[] table with size 7 which is equal to length of the Pattern

Step 1 - Define variables i & j. Set i = 0, j = 1 and LPS[0] = 0.



i = 0 and j = 1

Step 2 - Campare Pattern[i] with Pattern[j] ===> A with B.
Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

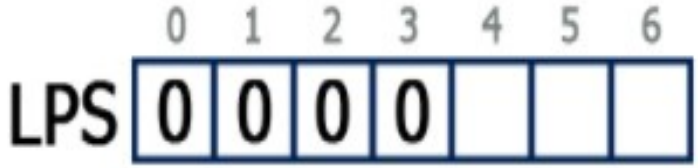


i = 0 and j = 2

Step 3 - Campare Pattern[i] with Pattern[j] ===> A with C.
Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

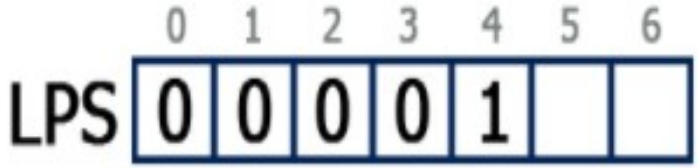
i = 0 and j = 3

Step 4 - Campare Pattern[i] with Pattern[j] ===> A with D.
Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.



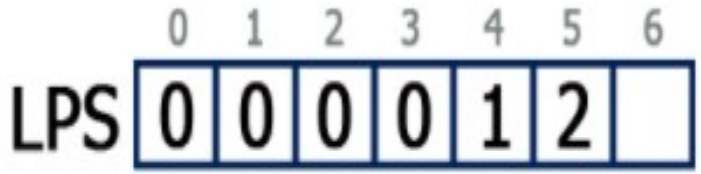
i = 0 and j = 4

Step 5 - Campare Pattern[i] with Pattern[j] ===> A with A.
Since both are matching set LPS[j] = i+1 and increment both i & j value by one.



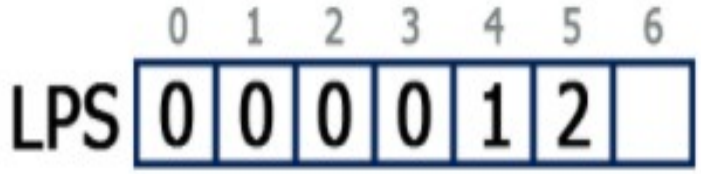
i = 1 and j = 5

Step 6 - Campare Pattern[i] with Pattern[j] ===> B with B.
Since both are matching set LPS[j] = i+1 and increment both i & j value by one.



i = 2 and j = 6

Step 7 - Campare Pattern[i] with Pattern[j] ===> C with D.
Since both are not matching and i !=0, we need to set i = LPS[i-1]
===> i = LPS[2-1] = LPS[1] = 0.



i = 0 and j = 6

Step 8 - Campare Pattern[i] with Pattern[j] ===> A with D.
Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	0

Here LPS[] is filled with all values so we stop the process. The final LPS[] table is as follows...

	0	1	2	3	4	5	6
LPS	**0**	**0**	**0**	**0**	**1**	**2**	**0**

Step 1: Create a one-dimensional array with a size equal to the Pattern's length. (LPS[size])

Step 2: Create variables i and j. Set i = 0, j = 1, and LPS[0] = 0 for i j, and LPS[0] respectively.

Step 3: Compare the Pattern[i] and Pattern[j] characters.

Step 4 - If both are equal, set LPS[j] = i+1 and add one to both i and j values. Step 3 is the next step.

Step 5 - Check the value of variable i if both aren't matched. If it's '0,' set LPS[j] = 0 and increase the value of 'j' by one; if it's not, set i = LPS[i-1]. Goto Step three.

Step 6- Repeat steps 1-5 until all of the LPS[] values are filled.

More Example

Example : T = b a c b a b a b a a b c b a b

P = a b a b a c a

The prefix value for each character can be tabulated as fallows.

A	B	A	B	A	C	A
0	0	1	2	3	0	1

More Example

When we compare P with T,

b a c b a b a b a a b c b a b

a b a b a c a

We get s = 4 and q (matched characters) =5

Next shift can be, S' = s + (q-k) where k=prefix function for last matched character.
In this example s' = 4 + (5-2) = 7 the position could be valid shift. We are avoiding the positions 5^{th} and 6^{th} as invalid by knowing prefix function value.

Rabin-Karp

- The Rabin-Karp string searching algorithm calculates a **hash value** for the pattern, and for each M-character subsequence of text to be compared.
- If the hash values are unequal, the algorithm will calculate the hash value for next M-character sequence.
- If the hash values are equal, the algorithm will do a Brute Force comparison between the pattern and the M-character sequence.
- In this way, there is only one comparison per text subsequence, and Brute Force is only needed when hash values match.

Rabin-Karp Example

Hash value of "AAAAA" is 37

Hash value of "AAAAH" is 100

1) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
   AAAAH
   37≠100 **1 comparison made**
2) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
    AAAAH
   37≠100 **1 comparison made**
3) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
     AAAAH
   37≠100 **1 comparison made**
4) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
      AAAAH
   37≠100 **1 comparison made**

...

N) AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAH
                                    AAAAH
6 comparisons made 100=100

Rabin-Karp Pseudo-Code

pattern is M characters long

hash_p=hash value of pattern
hash_t=hash value of first M letters in
body of text

```
do
  if (hash_p == hash_t)
    brute force comparison of pattern
    and selected section of text
  hash_t = hash value of next section of
           text, one character over
while (end of text or
       brute force comparison == true)
```

Rabin-Karp Complexity

- If a sufficiently large prime number is used for the *hash function*, the hashed values of two different patterns will usually be distinct.
- If this is the case, searching takes O(N) time, where N is the number of characters in the larger body of text.
- It is always possible to construct a scenario with a worst case complexity of O(MN). This, however, is likely to happen only if the prime number used for hashing is small.